VR Chemistry Lab - Handbook

July 18, 2026

Contents

1 Introduction

This handbook is intended to help people use the VR Chemistry Lab, both users of the software and people intending to expand it.

Users can find an installation guide and further information at How to use

Expanding the software by adding more reactions is highly encouraged. To get started with expanding the VR Chemistry Lab see How to expand

There are also the *VR Chemistry Lab – WebSocket API Reference* and *VR Chemistry Lab – System Architecture* for further programming information.

2 How to use

2.1 Installation

- Download the latest version (build) as a .zip from github.com/VRChemistryLab/VR_Chemistry_Lab
- Unpack the .zip folder
- Set up a VR-headset with PC-link
- Run *start.bat*

2.2 Features

Things you can do in the VR Chemistry Lab:

- use tools like tweezers, tubeclamps, test tubes and many more
- heat with a bunsen burner
- modular reactions
- multiple chemistry-engines + open to extention

Currently available reactions:

- 2 Na + I2 → 2 NaI
- 2 Na + Br2 → 2 NaBr
- 2 Na + Cl2 → 2 NaCl

2.3 Controls

The controls of the VR Chemistry Lab are listed in Table 1.

Action	**Control**
Move up/down	Right joystick
Turn left/right	Left joystick
Pick up an object	grip buttons
Interact with an object	trigger buttons

Table 1: Control scheme of the VR Chemistry Lab.

All objects on the tool cart located to the left of the laboratory table can be picked up. Whenever an object is within reach, it is highlighted to indicate that it can be picked up.

Objects that can be interacted with are listed in Table 2.

Object	Effect
Light remote	Turns all lights in the room on or off.
Tweezers	Pinches objects. Hold the interaction trigger to keep the tweezers closed.
Tube clamp	Opens or closes the clamp to hold or release a test tube. Once a test tube is clamped, the interaction trigger no longer needs to be held. Press the trigger again to release it.

Table 2: Interactive objects and their functionality.

2.4 Intended Reaction Process

Disclaimer: The VR Chemistry Lab is intended as a didactical tool. Its use by students should always be supervised and contextualized by a chemistry teacher.

For the salt synthesis reactions involving sodium and the halogens (I2, Br2, and Cl2), the intended procedure is as follows:

1. Heat a piece of sodium(Na) over the Bunsen burner using a test tube with a hole, held by a tube clamp.

2. Remove the lid of the selected halogen gas cylinder and drop the entire test tube into the cylinder.

3 How to expand

3.1 Prerequisites

For adding new objects only Godot is required. When adding a new reaction the tools below will be needed. (Depending on the reaction eather Cantera or Reaktoro)

The versions below are what this handbook and the current build were written against. Newer versions will likely work too, but if something in this section behaves differently than described, checking your version against this list is a good first step.

- **Godot** 4.5.1 – `https://godotengine.org/`
- **Python** 3.14 – `https://www.python.org/`
- **Cantera** 3.2.0 – `https://cantera.org/`
- **Reaktoro** 2.13.0 – `https://reaktoro.org/`

Cantera and Reaktoro are both installed via Conda, in separate environments – see Building the Reaktoro scripts for why and how. *PyInstaller* is additionally required for building either engine into an executable (`https://pyinstaller.org/`).

3.2 Adding new objects

3.2.1 Adding a new pickable object

The following steps are a manual to add objects that can be picked up in VR. Objects that are not meant to be interactable can be added as usual.

1. In a new scene, press *"Command+Shift+A"* or *"Instantiate Child Scene"*
2. Navigate to `res://addons/godot-xr-tools/objects` and select *"pickable.tscn"*, this is the basis for all VR pickable objects
3. Rename the root node *"PickableObject"* to <`object name`> and save the scene to the correct folder
4. Select the root node, in the Inspector, set *"Release Mode"* to *"Unfrozen"*
5. Add a *"MeshInstance3D"* and adjust the existing *"CollisionShape3D"* as fit

The object is now functional and can be picked up in VR.
For more configurations refer to **Adding GrabPoints to an object** and **Adding a highlight for objects in pick range**

3.2.2 Adding GrabPoints to an object

Adding GrabPoints to an Object is optional.
A working **pickable Object** is required.

GrabPoints will determine *how* the Object will be picked up by the VR Hand.

1. In the object scene, press *"Command+Shift+A"* or *"Instantiate Child Scene"*
2. Search for and add *"grab_point_hand_right.tscn"* and *"grab_point_hand_left.tscn"* to the object. Make sure the *"addon"* toggle is enabled.
   The nodes are hidden on default, enable visibility before continuing to the next step
3. Select either GrabPoint, in the Inspector under *"XRToolsGrabPointHand"* click on *"Hand pose"* and add a new *"XRToolsHandPoseSettings"*
4. Save the scene
5. Click on *"XRToolsHandPoseSettings"* to open the configuration tab
6. Drag and Drop or Quick Load a fitting hand pose from *"res://addons/godot-xr-tools/hands/animations/"* into both *"Open pose"* and *"Closed pose"*.
   Use the same pose for *"Open pose"* and *"Closed pose"* !
   Make sure to use the left poses for the left GrabPoint and vice versa, otherwise the pose will not work correctly.
   Pictures of the given hand poses: `https://godotvr.github.io/godot-xr-tools/docs/hand_poses/`
7. Adjust the GrabPoint transform as fit
8. Repeat steps 3. to 6. for the other GrabPoint
9. Disable visibility for both GrabPoints

When picked up, the object will now always assume the configured position.

3.2.3 Adding the snapContainer script to objects with SnapZone nodes

1. If the object has a *"SnapZone"* node, attach the script *"snapContainer.tscn"* to the root node

This will fix collision errors that are common with the SnapZone.

3.2.4 Adding a highlight for objects in pick range

Adding a highlight is optional.
A working **pickable Object** is required.

When an object is within pick range of the VR Hand, the objects gets highlighted with a bright colored border to help the user pick up objects more easily.

1. In the object scene, press *"Command+A"* or *"Add Child Node"*
2. Search for and add *"XRToolsHighlightVisible"* to the object.
3. Copy the Mesh Node of the object and paste it as a child node of the *"XRToolsHighlightVisible"*
4. Select the Mesh, in the Inspector, right click on *"Mesh"* and select *"Make Unique"*
5. In the Mesh, Drag and Drop or Quick Load *"highlight.tres"* in `"res://materials_sprites_and_textures`
into *"Material"*

3.2.5 Adding collision sounds

1. In the object scene, press *"Command+Shift+A"* or *"Instantiate Child Scene"*
2. Search for and add *"audio_collision.tscn"* to the root node.
3. Edit the *"AudioStreamPlayer3D"* to fit

NOTE: a configured scene for glass sounds has already been premade: `"res://objects/audio_nodes/audio_collision_glass.tscn"`

3.2.6 Adding spawners

The following steps are a manual to add a spawner for an object to put on the tool table.

1. In a new scene, add a *"Node3D"*
2. Rename the root node *"Node3D"* to `"spawner_<object name>"` and save the scene to the designated folder
3. Under `Node -> Groups`, check *"tool"*
4. Right-click on the root node, select *"Attach script"*, navigate to *"res://scripts"* and load *"spawner.gd"*
5. Select the *"spawner_<object name>"* node, in the Inspector, Drag and Drop or Quick Load the corresponding object scene into *"Spawned Object"*

6. Click *"Open Scene"* in the Inspector
7. In the spawned object scene, press *"Command+Shift+A"* or *"Instantiate Child Scene"*
8. Search for and add *"spawn_activator.tscn"* to the root node.
9. Back in the spawner scene, fill in *"Max Instances"* in the root node Inspector, this defines how many instances of that object can exist simultaneously at a time
10. Add a *"MeshInstance3D"* to the root node.
    (a) Copy the Mesh of the spawned object, scale down one axis to create a flat silhouette of the object

    or

    (b) For simpler silhouettes, use a primitive Mesh (e.g. *"CylinderMesh"*, *"BoxMesh"* etc.)
11. In the Mesh, Drag and Drop or Quick Load *"spawner_silhouette.tres"* in `"res://materials_sprites_and_textures/materials/"` into *"Material"*
12. Open the scene *"tooltable.tscn"* in `"res://objects/furniture/workstation"`
13. Select the *"spawners"* node
14. Press *"Command+Shift+A"* or *"Instantiate Child Scene"*
15. Search for and add the created spawner scene
16. Adjust the spawner transform as fit

3.3 Adding new chemicals

3.3.1 Adding a new chemical

The following steps are a manual to add a solid, pickable chemical, such as the existing Na. For other chemical types refer to **bottle stored chemical** and **standingcylinder stored chemical**

1. Follow the steps to create a **pickable Object**
2. In the object scene, press *"Command+Shift+A"* or *"Instantiate Child Scene"*
3. Search for and add *"reaction_part.tscn"* to the root node.
4. Select the ReactionPart, in the Inspector, fill in *"Chemical Name"*

5. Add a *"Area3D"* to the root node.

6. Add a *"CollisionShape3D"* to the *"Area3D"*, adjust the *"CollisionShape3D"* to be the same as the existing one.

7. Select the *"CollisionShape3D"* node, add it to the *"chemical"* group.

8. If the new Chemical is a precipitation:
   a) give it the *precipictation* tag in groups
   b) the mesh needs to be the size of 1mol of the chemical

3.3.2 Adding a bottle stored chemical

The following steps are a manual to add a bottle stored chemical, such as the existing bottle_na. For other chemical types refer to **chemical** and **standing-cylinder stored chemical**
A working **chemical** is required

1. In a new scene, press *"Command+Shift+A"* or *"Instantiate Child Scene"*

2. Navigate to `res://objects/base_objects` and select *"bottle.tscn"*, this is the base empty chemical bottle

3. Rename the root node *"bottle"* to `"bottle_<chemical name>"` and save the scene to the designated folder

4. Select the *"spawner_chemical"* node, in the Inspector, Drag and Drop or Quick Load the corresponding chemical scene into *"Spawned Object"* and fill in *"Chemical of this Bottle"* with `<chemical name>`

5. Follow the steps to create a **label** for the bottle

6. Select the *"decal_label"* node, in the Inspector, Drag and Drop or Quick Load the corresponding label png into *"texture_albedo"*

3.3.3 Adding a standing cylinder stored chemical

The following steps are a manual to add a chemical stored inside a standing cylinder. For other chemical types refer to **chemical** and **bottle stored chemical**

1. In a new scene, press *"Command+Shift+A"* or *"Instantiate Child Scene"*

2. Navigate to `res://objects/base_objects` and select *"standingcylinder.tscn"*, this is the base empty standing cylinder

3. Rename the root node *"standingcylinder"* to `"standingcylinder_<compound name>"` and save the scene to the designated folder

4. Press *"Command+Shift+A"* or *"Instantiate Child Scene"*

5. Search for and add *"standingcylinder_content.tscn"* to the root node
6. Right-click the node and select *"Make Local"*
7. Rename the node *"standingcylinder_content"* to <`compound name`>
8. Select the *"ReactionPart"* node, in the Inspector, fill in *"Chemical Name"* with <`compound name`>
9. Select the Mesh, in the Inspector, right click on *"Mesh"* and select *"Make Unique"*
10. In the Inspector, right click on *"Material"* and select *"Make Unique"*
11. Adjust the material to fit.

3.3.4 Adding a new label

1. Open the template file *"label template element.xcf"* or *"label template compound.xcf"* from `"Assets/labels"` in GIMP
2. Edit the template to fit and export it to Godot

3.4 Adding a new reaction

First decide which chemistry engine works best for the reaction.
Right now there are Cantera and Reaktoro to choose from.

Example reaction:

$$\text{Reactant1} + 2\ \text{Reactant2} \rightarrow 3\ \text{Product1} + \text{Product2}$$

is called `Reactant1Reactant2` everywhere.

3.5 Adding a new Cantera reaction

To Do

- Make the new reaction file (Python)
- Possibly add the new chemicals to Godot
- Register the reaction in the Godot reaction registry

3.5.1 Create the new reaction file (Python)

Location: `code/backend/Cantera`

1. Copy `simulation_Template.py`.
2. Rename it to `simulationReactant1Reactant2.py`
3. Replace all mentions of: Reactant1, Reactant2, Product1 with the actual names of the reaction participants.
4. Add fitting data everywhere marked with `#addDataHere`

What goes into `#addDataHere` The template needs two categories of data: thermodynamic data for every species involved (used by Cantera to compute enthalpy/entropy at the simulated temperatures) and kinetic data for the reaction itself (Arrhenius parameters, see the testing step below).

3.5.2 Create the corresponding YAML reaction definition

Location: `code/backend/Cantera/reaction_definitions`

1. Copy `template.yaml`.
2. Rename it to `Reactant1Reactant2.yaml`
3. Replace all mentions of: `Reactant1, Reactant2, Product` with the actual names of the reaction participants.
4. Add data everywhere marked with: `#addDataHere`

Notes

- Chemical data for the reaction participants can often be found at:

  `https://janaf.nist.gov/`

- Data for the reaction itself is often unavailable online. Testing is therefore required to ensure the simulation behaves correctly.

3.5.3 Test the reaction parameters A, E_a, and *site_density*

There is a python script specifically for testing these parameters, it can be found at: `code/backend/Cantera/tools/run_sweep.py`

The resulting plots have to be reviewed manually.

Usage run
`python tests/run_sweep.py simulationReactant1Reactant2.py [options]`
in the terminal from `code/backend`

CLI Flags

```
--single / --no-single
--pairwise / --no-pairwise
--triple / --no-triple
--all
--output PATH
--steps N
--temps T1 T2 ...
--time T
--baseline KEY=VAL ... (Example: --baseline Ea=500)
--values  KEY=VAL ... (Example: --values A=1e8,5e8,1e9 Ea=1e-4)
```

Examples

```
python tests/run_sweep.py simulationNaBr2.py --triple

python tests/run_sweep.py simulationNaCl2.py --single --pairwise

python tests/run_sweep.py simulationNaBr2.py --all --steps 4

python tests/run_sweep.py simulationNaBr2.py --triple --output comparisonPlots/mySweeps

python tests/run_sweep.py simulationNaBr2.py --all --temps 600 800 1000 --time 2.0

python tests/run_sweep.py simulationNaBr2.py --single --yaml path/to/reaction.yaml

python tests/run_sweep.py simulationNaBr2.py --triple --values A=1e8 Ea=1e4
    site_density=1e-3
```

The last example generates only one image using exactly those values.

Required in the Target Script

- Filename: `simulationReactant1Reactant2.py`
- Class name: `Reactant1Reactant2Simulation`
- The class must store: `self.YAML_PATH` in `__init__()`.

Optional Script Variables These parameters can be specified in the python script as well.

```
GAS_SPECIES         : list
SURFACE_SPECIES     : list
SWEEP_RANGES        : dict
MAX_TIMESTEP        : float
START_TEMPERATURES  : list
```

```
SIMULATION_TIME     : float
OUTPUT_BASE         : str
TMP_YAML_DIR        : str
RUN_SINGLE          : bool
RUN_PAIRWISE        : bool
RUN_TRIPLE          : bool
```

Optional Default Values

```
START_TEMPERATURES = [400, 600, 800, 1000, 1200]
SIMULATION_TIME    = 1.0
MAX_TIMESTEP       = 1e-3
```

SWEEP_RANGES Format

```
SWEEP_RANGES = {
    "site_density": {
        "start": 1e-5,
        "end": 1e-3,
        "steps": 5,
        "log_scale": True
    },
    "A": {
        "start": 1e8,
        "end": 1e10,
        "steps": 5,
        "log_scale": True
    },
    "Ea": {
        "start": 10,
        "end": 1e4,
        "steps": 5,
        "log_scale": False
    }
}
```

3.5.4 Finalize the YAML file

After evaluating the sweep results, add the most realistic combination of parameters to the YAML file.

3.5.5 Possibly add the new chemicals to Godot

If the chemicals used in the reaction are not integrated in the Godot-Project yet, they need to be added for the reaction to work in VR.

For this, refer to the handbook section: **Adding a new chemical**

3.5.6 Register the reaction in the Godot reaction registry

`reaction_registry.json` is **generated, not hand-written**.

Location: `code/backend/tools/ReactionRegistry_Generator.py`

1. **Optional:** add a line to `code/backend/tools/reaction_meta.yaml` with the new reaction's id and a rough glow intensity, e.g.:

   ```
   Reactant1Reactant2: 0.5
   ```

   This is the *only* piece of data that isn't auto-discovered. If skipped, the reaction defaults to `reactionStrength: 0.0` (no glow) – nothing breaks, it just won't glow until someone adds a line here.

2. **Run the generator** (from anywhere – its paths are resolved relative to the script itself, not the current directory):

   ```
   python code/backend/tools/ReactionRegistry_Generator.py
   ```

   This rewrites `godot_project_VR_Chemistry_Lab/data/reaction_registry.json` from scratch: it scans `backend/Cantera` for every `simulation*.py` file to get the reaction id and engine, and reads the matching `reaction_definitions/<Id>.yaml` to fill in reactants/products straight from the reaction equation.

3. Restart the Godot project so `DataSynchronizer` – which currently only loads the registry once at startup – picks up the new entry.

Note: forgetting to re-run the generator after adding a new reaction is the most likely reason a "finished" reaction still doesn't show up in VR – `DataSynchronizer.getEngineConnectionForReaction` simply won't know about it until `reaction_registry.json` is regenerated.

3.5.7 Adding the changes to the executable

When new files have been added, they need to be added to the executable too, in order for the VR App to include their behavior.

For this, refer to the handbook section: **Adding Cantera files to the build**

3.6 Adding a new Reaktoro reaction

Reaktoro has no per-reaction file at all. One `ReaktoroEquilibriumEngine` instance computes whatever equilibrium a given element space can reach (Gibbs-energy minimization via `SupcrtDatabase`) – so "adding a reaction" for Reaktoro really just means teaching the shared engine one more reagent; any combination of known reagents then reacts automatically, with no per-combination code.

Location: `code/backend/Reaktoro/reaktoro_engine.py`

1. **Add the reagent** to the `REAGENT_FORMULAS` dict: display name (must match `ReactionPart.CHEMICAL_NAME` in Godot) → chemical formula, e.g.:

```
REAGENT_FORMULAS = {
    ...
    "MgSO4": "MgSO4",
}
```

2. **Check the element space.** Every element in the new formula must appear in the `ELEMENTS` string at the top of the file (currently `"H O C Na Cl Ca S N K Mg Fe Cu Ba Pb"`). If not, add it – otherwise Reaktoro silently can't form the species and `state.add(...)` for it is skipped.

3. **Check the gas phase, if relevant.** If the new reagent could plausibly produce a gas, make sure it's listed in `gasPhase` in `ensureBuilt()` (currently `H2O(g) CO2(g) H2(g) O2(g) N2(g)`). Otherwise that gas is simply invisible to the client – no `gas_mol_*` entry, no `bubble_rate` contribution – it doesn't break anything, it just won't bubble.

4. Possibly add the new chemical to Godot, using the same display name as in step 1.

5. **Re-run the registry generator** (same script as for Cantera reactions). This reads `REAGENT_FORMULAS` straight out of `reaktoro_engine.py` via `ast` and writes its keys into `reaction_registry.json`'s top-level `aqueousReagents` list – that's what lets Godot recognize the new chemical as aqueous and route it to the `"WebSocket Reaktoro"` engine via the shared `"Aqueous:"` entry.

6. If running from a build rather than from source, rebuild the Reaktoro executable (Building the Reaktoro scripts) so the change is actually included.

3.7 Making a new build

A build is an executable version of the project. For changes in the source code to take effect, oftentimes a new executable needs to be made.
It is possible to make a completely new build of both the server and the godot project, as well as only building one and keeping the other, if nothing changed on that side.

Either way the structure needs to look like this:

VRProjekt/ ← this can be renamed

```
|-- start.bat   ← the thing you click on to run
|-- licenses/
|-- server/
|   |-- cantera/
|   |   |-- websocket_server.exe
|   |   |-- simulation*.py
|   |   |-- reaction_definitions/
|   |-- reaktoro/
|   |   |-- reaktoro_websocket_server.exe
|-- godot/
    |-- libgodotopenxrvendors.dll
    |-- VRApp.console.exe
    |-- VRApp.exe
    |-- VRApp.pck
```

3.7.1 Building the Godot project

If something in Godot was changed, a new build from the Godot project needs to be made.
For this, the fitting export template needs to be installed in Godot. For PC-VR from a Windows-PC this is the windows export template. Then export the godot project (to windows executable). For VR to work *openxr* needs to be added in the features tab.
For the *start.bat* to recognise the godot build it needs to be called `VRApp` and the folder structure needs to be according to the figure in Making a new build

3.7.2 Building the Cantera scripts

If only a simulationReactant1Reactant2 script has been changed this step can be skipped, continue with adding the changed files to the build.

If other Cantera specific files have been added or changed continue here.

The python package *PyInstaller* is required for this step. It can be found at `pyinstaller.org`.

It is recommended to build Cantera and Reaktoro from separate conda environments (see Building the Reaktoro scripts for why). Activate (or create) the Cantera environment first, e.g.:

```
conda activate ct-env
```

With pyInstaller installed open the terminal in `code/backend/Cantera` and run
`pyinstaller --onefile --hidden-import=cantera --hidden-import=websockets --collect-all cantera websocket_server.py`

This creates a *dist* folder with the `websocket_server.exe`.

Put this in the `server/cantera` folder of the build.

Then potentially add the other Python files to the build

3.7.3 Adding Cantera files to the build

Copy all files with the name like *simulationXYZ.py* and the whole folder `reaction_definitions/` from `code/backend/Cantera` into `build/server/cantera`.

3.7.4 Building the Reaktoro scripts

Why a separate conda environment Cantera and Reaktoro are both distributed exclusively through the `conda-forge` channel (Reaktoro in particular has no reliable `pip` package – it must be installed via Conda). Installing both into the same environment risks pulling in conflicting versions of shared native dependencies, and since PyInstaller bundles whatever is active in the current environment, a broken or oversized executable is hard to diagnose after the fact. Keeping one environment per engine avoids this entirely and mirrors how the two servers are already kept in separate folders in the build.

Setting up the environment

```
conda create -n reaktoro-env python=3.11
conda activate reaktoro-env
conda install -c conda-forge reaktoro
pip install websockets pyinstaller
```

(`websockets` and `pyinstaller` are regular PyPI packages and can be installed with `pip` inside the conda environment; only `reaktoro` itself needs to come from `conda-forge`.)

Building With the `reaktoro-env` environment active, open the terminal in `code/backend/Reaktoro` and run:

```
python -m PyInstaller --onefile --hidden-import=reaktoro --hidden-import=websockets
    --collect-all reaktoro reaktoro_websocket_server.py
```

This creates a *dist* folder with the `reaktoro_websocket_server.exe`. Put this in the `server/reaktoro` folder of the build.

3.8 Adding a new chemistry simulation server

The server code lives in `code/backend/` in its own folder (see `Cantera/` and `Reaktoro/` for the existing examples). Adding a third engine means implementing a new server that speaks the same wire protocol as the existing ones and hooking it into both the build and the Godot registry.

1. **Create the backend folder**, e.g. `code/backend/<EngineName>/`, following the same shape as `Cantera/`: a dispatcher (analogous to `simulation_hub.py`) that resolves a reaction name to a simulation class, and a `<engine>_websocket_server.py` that serves it over WebSocket.

2. **Implement the wire protocol.** The new server must accept the same request shape and return the same response envelope (`success`/`reactionInstanceId`/`reactionIds`/`data`/`error`) and the same method catalog (`init`, `runUntilTargetTime`, `startHeatingAndRunUntilTargetTime`, `stopHeatingAndRunUntilTargetTime`, `closeSession`) – see the *VR Chemistry Lab – WebSocket API Reference* document for the exact contract. Godot's `server_handler.gd`/`json_writer.gd`/`json_parser.gd` are engine-agnostic and do not need to change for a new engine, as long as this contract is respected.

3. **Pick a port** for the new server and run it alongside the existing ones (each engine currently gets its own WebSocket port, e.g. Cantera on `50505`).

4. **Add an Engine node in Godot.** Under the *"Engines"* node add a new *"WebSocketClient"* child pointing at the new server's host/port, so `DataSynchronizer` can resolve it.

5. **Register reactions for the new engine** in `reaction_registry.json`, same as in Register the reaction in the Godot reaction registry, using the new engine's id.

6. **Add a build step**, analogous to Building the Reaktoro scripts: its own conda environment, its own PyInstaller command, its own `server/<engine>/` folder in the build.

3.9 Further expansion possibilities

There are many more possible directions for expansion, enabled by different design decisions.

- storing chemicals chemically correct → option to, e.g. add a drop of fluid bromine and then heat to make gaseous
- standalone VR by setting up a server for the chemistry → The python code runs completely decoupled on a websocket server. Add the servers address in the inspector to the *"webSocket"* host field.
- multiplayer
- a molecular-view, by adding a differend kind of chemical server